



ros2_control extra bits

Ros2_control Extra Bits



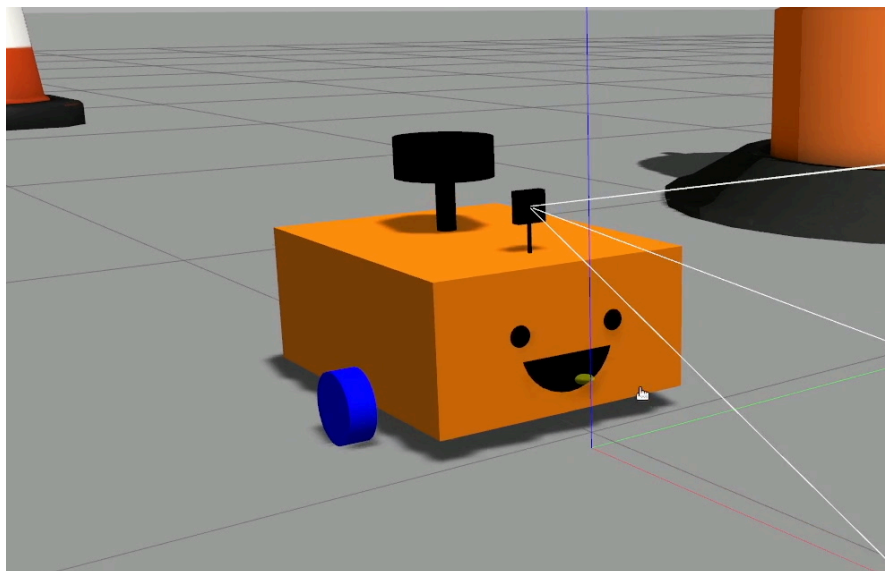
There are a few follow-up points regarding `ros2_control` for our differential drive robot that are worth sorting out before we move ahead with the rest of the tutorial series. I've kept these out of the main video to avoid further detracting from the core explanation of how to get `ros2_control` up and running.

Update URDF to match robot structure

The first step is to update your URDF to match your physical robot (if you have one). In my case I did the following:

- Set up `xacro` parameters for all the key dimensions and set them accordingly
- Changed my Gazebo and RViz colours to match the real robot
- Fixed up the controller YAML to have the correct wheel separation and radius
- Added an extra `xacro` file for a smiley face on the front, like my real one currently has (it will eventually have a screen)

You can check out the differences [here](#).



Problem 1 - Gazebo clock rate

At the end of the previous tutorial I mentioned that there are a few problems with the simulation. The first of these is that you'll notice the robot motion in RViz is quite "choppy" while driving the robot around. That's because the Gazebo-ROS clock is only running at 10Hz, so the rest of ROS gets locked to that slow speed. We need to tell Gazebo to publish the clock faster. This isn't only a visual thing, it's actually causing our odometry to drift quite badly (but don't check that now because there are other problems affecting it too which we'll fix soon).

Fixing the clock rate is a bit annoying. First we need to create another parameter file, a good name is `gazebo_params.yaml`. In it we'll put the following code, which is telling `gazebo` to publish the clock at 400Hz instead of 10Hz (if your computer is slow you may need to reduce this).

```
gazebo:
  ros__parameters:
```

```
publish_rate: 400.0
```

We now need Gazebo to see these parameters when it starts up. In our `launch_sim.launch.py` we are "including" the ROS-provided Gazebo launch. This launch file has an argument called `extra_gazebo_args` which lets us just send whatever we want down to the Gazebo it runs. In this case, we'll create a variable to store the path of our params file (similar to how we handled the URDF), and then combine that path into a string telling Gazebo to read it.

```
gazebo_params_path = os.path.join(
    get_package_share_directory(package_name), 'config', 'gazebo_params.yaml')

gazebo = IncludeLaunchDescription(
    PythonLaunchDescriptionSource([os.path.join(
        get_package_share_directory('gazebo_ros'), 'launch', 'gazebo.launch.py')]),
    launch_arguments={ 'extra_gazebo_args': '--ros-args --params-file ' + gazebo_params_path }.items()
)
```

Note that [future versions](#) should let us pass the params file in directly as a launch argument, simplifying this process ever so slightly.

If we rerun Gazebo now we'll see that RViz is much smoother.

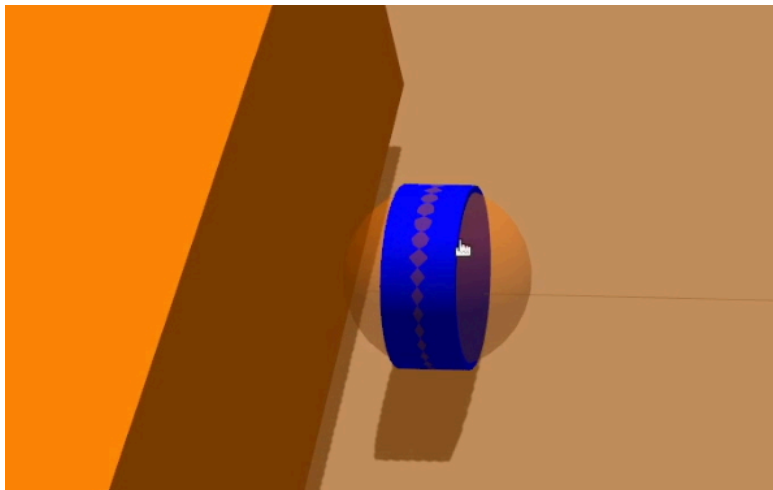
On my computer I also get an error printed in the Gazebo terminal which is a bit annoying, I think the Gazebo command line parser is treating the file as an extra SDF to load and failing. We can just ignore it, but if you've got a fix then let me know!

Problem 2 - Wheel drift

The next problem we have is wheel drift. The easiest way to spot this is to do a fresh start on Gazebo/RViz, and do a full revolution of the robot. We should see that when we come back to the initial angle in Gazebo, RViz is no longer in sync.

There could be a few factors affecting this, but an easy one to tackle is that the wheel collision is a cylinder, which means there will be subtle skidding effects occurring due to the ground connection being a line. We could somewhat alleviate this by reducing the thickness of the wheel collision cylinder, but better is to reduce it infinitely, by replacing the cylinder with a sphere. Now the contact with the ground will be a single point per wheel.

All we need to do is in our left/right wheel links of our URDF, in the `collision` geometry (not visual!) replace `cylinder` with `sphere` and remove the `length` parameter. To check this we can enable collision view in Gazebo to see the collision sphere and the visual cylinder.



There may be better methods than this - if you have one please let me know!

Additionally, if you are still having issues with wheels skidding and drifting oddly, try experimenting with the friction parameters like we did for the caster.

If we relaunch it we should see that a full rotation places us back where we started.

For what it's worth, the reason this wasn't a problem with the old plugin is that it actually cheats to provide the odom transform perfectly rather than calculating it based on the simulated wheel spin.

Problem 3 - Sensor synchronisation issue

Now it's time for problem number 3, which I currently don't have a fix for, and if someone can figure out what's going on, I'll be very grateful. I have a [whole GitHub repo](#) and separate video for reproducing it.

Gazebo ros2_control diff drive issue



It's difficult to describe in text (just watch the video) but essentially, when using `ros2_control` with Gazebo the sensor output appears to be out of sync with the published transforms.

I don't think it's a friction issue, as I have tried to eliminate that. It seems more like a timing issue, but the odd thing is that the laser data moves in the direction of rotation, not away from it, suggesting that the transforms are being published *earlier* than the sensor data rather than later (so it's not just be a delay in `ros2_control`).

It could be to do with acceleration limits, I have tried to experiment with this too with no success. Keeping accelerations and velocities low reduces the effect, which may be useful in testing.

Swapping between control methods

Given that we have these issues, we might sometimes want to swap back to the old control method. It generally tends to work a bit more "perfectly" which is not always what we want but can be handy.

Taking the steps below will let us easily swap between the Gazebo diff drive plugin and the `ros2_control` one.

Add xacro argument & condition

Using xacro to build our URDF gives us the ability to pass in an argument at runtime to change the result. This could be anything like the length of an arm, the colour of a wheel, but in this case we want to pass in a boolean to tell it whether to use `ros2_control` or not.

We'll start by adding an argument declaration at the start of our `robot.urdf.xacro`, and default it to true (we generally do want to use `ros2_control`).

```
<xacro:arg name="use_ros2_control" default="true"/>
```

Now we need to add a conditional to change which bit of the URDF is used. Xacro doesn't have an `if-else` construct that we may be familiar with, instead it has `if` and `unless` which will trigger depending on whether the condition is true, or untrue. So down where we have our `gazebo_control.xacro` commented out, we instead want to add the following blocks:

```
<xacro:if value="$(arg use_ros2_control)">
  <xacro:include filename="ros2_control.xacro" />
</xacro:if>
<xacro:unless value="$(arg use_ros2_control)">
  <xacro:include filename="gazebo_control.xacro" />
</xacro:unless>
```

So, if `use_ros2_control` is true, include the `ros2_control.xacro` file, and unless `use_ros2_control` is true, include `gazebo_control.xacro`.

Pass in the xacro argument

Now we need to pass the xacro argument in when we process the file, which is in our robot state publisher launch file (`rsp.launch.py`).

This *should* be straightforward but actually isn't. Right now this file is using the `xacro` Python module to process the file, which is all well and good, and it has the option (*don't type this!*) to set arguments by specifying the mappings like so:

```
xacro.process_file(xacro_file, mappings={'use_ros2_control': 'true'})
```

This is fine, and works as is, but ideally we want to be able to set it at the level above this, to pass it in as a *launch argument* and because of how launch files are processed, dropping a `LaunchConfiguration` variable in here like we did with `use_sim_time` just won't work.

The workaround is a little hacky but it's not too bad, especially considering launch scripts aren't designed to be super python-heavy anyway. Instead of using the xacro Python module we're going to call the xacro process using the launch `Command` substitution.

We'll go ahead and make the launch argument anyway, right under our `use_sim_time` one.

```
use_sim_time = LaunchConfiguration('use_sim_time')
use_ros2_control = LaunchConfiguration('use_ros2_control')
```

Next, to make things smoother in case we ever want to go back to the old one, we'll move the `toxml()` up to the process call. Hopefully you can see that that won't really change anything except that `robot_description_config` will now be the final string (rather than some `xacro` object), which makes more sense anyway.

```
## Was
robot_description_config = xacro.process_file(xacro_file)
params = {'robot_description': robot_description_config.toxml(), 'use_sim_time': use_sim_time}

## Now
robot_description_config = xacro.process_file(xacro_file).toxml()
params = {'robot_description': robot_description_config, 'use_sim_time': use_sim_time}
```

So the `params` is now taking in a string. But because it is a part of the clever ROS Launch system, instead of a string it can also take a *substitution*. So we'll take advantage of the `Command` substitution, and replace the `robot_description_config` assignment to a call out to the `xacro` process rather than the Python module. Make sure you also add the appropriate `import` at the top.

```
# At the top
from launch.substitutions import Command

# Down where we were
```

```
#robot_description_config = xacro.process_file(xacro_file).toxml()
robot_description_config = Command(['xacro ', xacro_file, ' use_ros2_control:=', use_ros2_control])
```

This line is just constructing a command that could be typed into a terminal, but instead it's going to run it inside the launch script and use the output. We can see that `use_ros2_control` (a `LaunchConfiguration` argument) is used in this call, so at runtime it will substitute that for whatever is passed in, then substitute the result of the whole command into the `robot_description_config` variable as a string.

Pass in the Launch argument

Then in `launch_sim.launch.py` where we were including our robot state publisher launch, we can set it to whatever we want by updating the `launch_arguments` line when including `rsp.launch.py`:

```
...
launch_arguments={'use_sim_time': 'true', 'use_ros2_control': 'true'}.items()
...
```

This is what I demonstrated in the video, however it would be nice to be able to toggle it at runtime, i.e. by running:

```
ros2 launch my_package launch_sim.launch.py use_ros2_control:=false
```

There are actually two ways to achieve this. The first is to...do nothing. If we **undo** the previous step where we passed down the launch argument, we'll find that we are able to set it at the command line and it will automatically find its way down to where it needs to be.

Alternatively, we can create another launch argument, this time in `launch_sim.launch.py` (probably with the same name), and pass it down like

```
use_ros2_control = LaunchConfiguration('use_ros2_control')

...

launch_arguments={'use_sim_time': 'true', 'use_ros2_control': use_ros2_control}.items()
...
```

These will have the same effect, but the latter is a bit more explicit. We should also remember to add an appropriate `DeclareLaunchArgument` entry.

This has probably all been a bit confusing, so check out the final files below.

`rsp.launch.py`:

[Click to show code](#)

`launch_sim.launch.py`:

[Click to show code](#)

RViz Config

One last thing I did was to set up RViz the way I liked it and saved the configuration to the `config` directory so that it can be loaded back up again in future with `rviz2 -d`.

Conclusion

Now that we've got those things tidied up a bit, we can **move forward** with `ros2_control` on our robot, and then start to run some algorithms!

**EricMack**

Sep '22

Hi Josh just watched the videos you posted dropped for module number 12. I'm getting really excited as I see all of this coming together. I've been collecting the bits and pieces to build my own set up and follow your tutorials.

My question has to do with the physical hardware controller that you used it looks like it's a gaming controller of some kind. I'm not very familiar as I don't have a gaming system or play one.

I realize that ultimately I could use most any gaming controller if I wanna just write my own inner face but for the moment I just wanna duplicate what you've done and then branch off on my own

Do you have any thoughts or recommendations on a particular one to buy and start with So I can use the exact configuration you are using?

[1 reply](#)

**JoshNewans**[▶ EricMack](#) Sep '22

Hey Eric, I'll be diving deeper into that in 2 videos time, but here's a few quick points 😊 :

- I'm using an old wired (USB) Xbox 360 controller
- There are generally 3 types of controllers: USB wired, Bluetooth, and wireless with a USB dongle
 - Wireless is obviously nicer, and means you can connect to either the robot or a development machine on the network, whereas wired kind of restricts you to the latter
 - I don't trust Bluetooth reliability, especially on Linux and on the Pi, so I prefer USB-based options, but many people do have success with Bluetooth (PlayStation controllers being a classic for this). It's also more annoying to re-pair etc.
- Depending on the controller you may or may not need Linux drivers for it (just as if you were playing a game with it). In my limited experience, USB wired ones usually work with no extra software or issues, wireless/Bluetooth can be trickier.
- Once the OS sees it as a "joystick" they are all pretty much the same as far as ROS is concerned. We use `joy/joy_node` to read the raw joystick data from the OS and publish it to `/joy`.
- Now that the joystick input is available as a ROS topic we can do whatever we want with it. For our differential drive robot, we use `teleop_twist_joy` to take those axis/button press measurements from `/joy` and convert them to a Twist message on `/cmd_vel`. The axis/button numbering will be different for different controllers, you just have to set the parameters accordingly (e.g. axis 1 is forward/reverse, axis 2 is left/right, button 4 is enable).
- So with all that being said **pretty much any controller/joystick/gamepad will work**. I personally just like the ergonomics of the Xbox controllers, and I'll see if I can get my wireless one (Xbox One S version w/ Bluetooth) going for the video, otherwise I usually just plug the wired one into a laptop and carry it around.
- In the video I'll also briefly cover connecting a gamepad to your phone, and then connecting the phone to ROS. For that I'll be using [this](#) Bluetooth phone gamepad, but I'd be hesitant to recommend it as I found the joystick deadzones too jarring.
- Thanks for prompting me to think about this. I've just ordered [this one](#) (eBay Australia link) as it seems like it might be a fairly common, cheap one, so hopefully it works ok and I can recommend it in future.

Hope that helps!

**EricMack**

Sep '22

Josh, thank you. Helpful information. I just purchased the same controller for \$13 USD, so I can test what you are doing. I think it is helpful when you post links to what parts you are using so others can follow-along.

-Eric

**EricMack**

Oct '22

Josh, I've been thinking about an idea for a future tutorial for you, related to robot control. It's one I would really like to see.

Once you have the ROS2 Control working with your joystick/game controller, I would like to see you add a control node using an Arduino, LCD shield, a few buttons and potentiometers. This would help us see how your Arduino node would do for I/O, say on a custom portable control pendant. Here's what I have in mind:

INPUTS

2 buttons - Honk Remote buzzer or flash LED headlights.

3 potentiometers - speed control, throttle (and, ideally, steering, perhaps using a dual pot joystick but I understand that is more involved).

OUTPUTS

Buzzer - when a bump switch on robot is activated it should trigger this buzzer/light/whatever

LCD - One line for Speed and heading (relative to anything you can grab); One line for angle of something, like the wheels; and One line for distance to object – whatever is in front.

What I am really keen to learn is how you would plan this and do the interface to both broadcast and listen to the appropriate pins/signals.

The concepts you develop here, perhaps with input from the community, would allow us to make custom controllers with bidirectional capabilities.

If you do not have an LCD shield, I would be happy to make a contribution to cover one.

I would really like to see this in a future video.

Thanks for all you do to teach and guide us.

Eric

Update: If the above is a task that is better suited for a Raspberry Pi, that is OK. If so, I would like to understand why (e.g. software, IO, libraries, etc. If Raspberry Pi, could we use a PICO W with an I2C Shield or something?). This is what I think an excellent concept that people [like me] could run with once they have an understanding of how to approach this. Thanks for considering.

[1 reply](#)

**JoshNewans**[▶ EricMack](#) Oct '22

Hey Eric, interesting idea!

Are you imagining something like the ones James Bruton has made for his videos?

He's had a few different ones (at least 3), but the "old one" shown at the start of [this video](#) seems to be what you're describing.

I had not considered it, but that's not a bad idea, I will give it further thought. I'd probably be leaning toward the Pico W (as you've suggested) and programming it with micro-ROS or CircuitPython, but those decisions would mostly be driven by me having never used those platforms before and it seems like a good starter project to get my feet wet.

[1 reply](#)

**EricMack**[▶ JoshNewans](#) Oct '22

Yes! I had forgotten about his video. I've had the idea in the back of my mind for too many years, but the missing link was the software. You are convincing me that ROS2 is the way to go.

Please do consider the basic proposal in one of your upcoming videos. Happy to help brainstorm.

UPDATE: I just watched the video again. I like his idea of using a touch screen for more virtual buttons. Might be cool someday to touch a map of the room and the robot goes there. Then a PTZ Camera on the robot can beam video back. He uses an Arduino to manage the I/O and send/receive commands via Serial to USB into the Pi, which runs the ROS node. Source code in his github but I think it may be ROS not ROS2.

**Niranjana_Rao**

Nov '22

Hi Josh,

Thanks for share this ros2 control video it is very helpful.

I am getting this error while running "ros2 launch my_bot launch_sim.launch.py". If you have any suggestions to resolve this issue. kindly please share it with me.

[gzserver-2] [ERROR] [1669334236.428722199] [gazebo_ros2_control]: parser error Couldn't parse params file: '-params-file /home/niranjan/dev_ws/install/my_bot/share/my_bot/config/my_control_params.yaml'. Error: Cannot have a value before ros__parameters at line 4, at /tmp/binarydeb/ros-foxy-rcl-yaml-param-parser-1.1.14/src/parse.c:623, at /tmp/binarydeb/ros-foxy-rcl-1.1.14/src/rcl/arguments.c:388

Thanks
Niranjan

[1 reply](#)



Niranjan_Rao

[► Niranjan_Rao](#) Nov '22

Hi Josh,

I think I am able to resolve this issue. It looks like the my_controls.yml parameter file ros__parameters requires two underscore I had a typo here. After fixing the typo the issue got resolved.

Thanks
Niranjan

[1 reply](#)



JoshNewans

[► Niranjan_Rao](#) Nov '22

Glad you were able to get it fixed!
And yes that double underscore is an easy mistake to make, I've definitely tripped on that before!

[1 reply](#)



Stress_Robotics

Dec '22

Hi I have a problem guys, after implementing ros2_control , the robot moves in the opposite direction (forward and backwards) in rviz2... Gazebo is working fine though (except for the publish_rate thingy)... please any ideas on how to solve this?

[2 replies](#)



JoshNewans

[► Stress_Robotics](#) Jan '23

Hmm, the first thing I'd say would be to step through and make sure you understand your kinematic chain, i.e. which axes are pointing in which direction. I believe I had Z pointing out, i.e. the left wheel is "positive-Z-rotation-forwards" and the right wheel is "positive-z-rotation-backwards".

The axis of the right wheel joint is then flipped, and you should be able to check this by not running ros2_control and instead using joint_state_publisher to manually control the joint angles. Make sure that a positive joint corresponds to the robot moving forward and vice versa.

Then in ros2_control1.xacro and my_controllers.yaml (or whatever you have called them), ensure that left and right are assigned correctly.

Finally, make sure that your fixed frame is odom in RViz.

Let me know how you go!

[2 replies](#)



mhered

[► JoshNewans](#) Feb '23

Hello Josh,

After implementing the fix to increase gazebo clock rate, I get the following error message when running gazebo - even though it otherwise seems to be working fine:

```
...
[gzserver-2] Error [parser.cc:403] Error parsing XML in file
[/home/mhered/dev_ws/install/manolobot_uno/share/manolobot_uno/config/gazebo_params.yaml]: Error document empty.
...
```

It appears it is trying to interpret the gazebo_params.yaml file as XML and failing.

I noticed also your comment in the blog post:

On my computer I also get an error printed in the Gazebo terminal which is a bit annoying, I think the Gazebo command line parser is treating the file as an extra SDF to load and failing. We can just ignore it, but if you've got a fix then let me know!

Is this the error you refer to?



Stress_Robotics

[► JoshNewans](#) Feb '23

I got it fixed. it was the kinematic chain problem. Thank you

[1 reply](#)



tiptonnet

Mar '23

So I've been following your tutorial for a project. Thank you BTW. It's been hard to find Raspberry Pi examples. Problem is. I ran this part of the tutorial last night and it all seemed to work with exception of a type in my config. The robot started doing flips when set to move forward. Anyways I think I got that solved but now, today when I try to run it the controller manager is not loading... All the sudden it's just not there any more.

Output:

```
ros2 run controller_manager spawner diff_cont
[INFO] [1679414845.079311127] [spawner_diff_cont]: Waiting for '/controller_manager' node to exist
[INFO] [1679414847.090650840] [spawner_diff_cont]: Waiting for '/controller_manager' node to exist
[INFO] [1679414849.100639643] [spawner_diff_cont]: Waiting for '/controller_manager' node to exist
[INFO] [1679414851.114171358] [spawner_diff_cont]: Waiting for '/controller_manager' node to exist
[ERROR] [1679414853.122073734] [spawner_diff_cont]: Controller manager not available
```

Why now? Worked last night. I can't even list interfaces... Same problem.

[1 reply](#)



tiptonnet

Mar '23

Okay... Somehow controller manager mysteriously started working. Thanks to Niranjan_Rao pointing out the underscore issue, that did make a difference.

But now he is back to doing flips instead of moving forward or reverse. I'm sure I'll figure it out, obviously a config issue.



Afshan_Navazdeen

Apr '23

Hi Josh,
Didn't find any issues until Ros2_control was introduced.

With gazebo_ros2_control my robot moves only forward and backwards, doesn't respond the angular velocity command and the /odom frame doesn't show up in the RVIZ fixed frame selection list.

I have tried it on Foxy and humble. the result is the same.

Any ideas how I can fix this.

[2 replies](#)



haoluoVietNam

May '23

B	JoshNewans Hi Josh, Your tutorial is very helpful. and i want to use ros_control to control my robot with ackermann type instead of your diff. You can make a video tutorial. I sincerely thank	May '20
A	Afshan_Navazdeen Further into the investigation i have noticed the odom TF has the below error. Error: TF NAN_INPUT: Ignoring transform for child_frame_id "base_link" from authority "Authority undetectable" because of a nan value in the transform (-nan -nan 0.000000) (-nan -nan -nan -nan) at line 278 in /tmp/binarydeb/ros-foxy-tf2-0.13.14/src/buffer_core.cpp Error: TF DENORMALIZED_QUATERNION: Ignoring transform for child_frame_id "base_link" from authority "Authority undetectable" because of an invalid quaternion in the transform (-nan -nan -nan -nan) @JoshNewans Can you please help me out on what causes this error and how to resolve it. This has practically stopped me in continuing further in to this wonderful tutorial. 1 reply	► Afshan_Navazdeen May '23
A	Alessandro_Gomez I'm having the same problem. Everything works except the angular velocity 1 reply	► Afshan_Navazdeen Jun '23
Z	Ziad_Shoeib Hello, I have been following along and thanks for your great efforts, everything had been working fine with me until this point. When I tried the laser scan at rviz with ros2_control, the scan moves totally with the robot, the dots move with the orientation of the robot not just wiggle as the small problem but it totally moves. When I switch to gazebo_control everything comes back to normal, not sure what the problem can be? 1 reply	Jun '23
A	Afshan_Navazdeen @Alessandro_Gomez Did you find a solution to this?	► Alessandro_Gomez Jun '23
V	vijay_sankar I am also facing the same issue. ros2 launch articubot_one launch_sim.launch.py everything is working fine. when i launch ros2 launch articubot_one launch_robot.launch.py i am having controller_manager issues . I did not connect my joystick. and my physical robot has no response.	► tiptonnet Jun '23
P	Phyo_Min_Thant FYI. In ROS2 Humble, I could perfectly run with just "spawner" not "spawner.py". It is a little change as you mentioned. 2 replies	Jul '23
V	vijay_sankar How did you fix the Hardware_interface issues in Humble using GitHub - joshnewans/diffdrive_arduino: A simple ROS Control Hardware Interface for differential drive control of robot whose motors are connected via an Arduino (or similar microcontroller). this Diffdrive_arduino . I was stuck here.	► Phyo_Min_Thant Jul '23
U	Uby Hi, I facing some issue with gazebo ros2 control package. I followed the instruction for the ros2 control and launch the launch_sim.launch.py everything work perfect but could not connect with gazebo_ros2_control The error msg: [gzserver-2] [ERROR] [1691414356.585613673] [gazebo_ros2_control]: Error parsing URDF in gazebo_ros2_control plugin, plugin not active : text not specified in the min tag because this I wasn't able to connect with hardware interface error msg: ros2 control list hardware_interfaces Could not contact service /controller_manager/list_hardware_interfaces Can you guys help me to figure out what mistake I have made. Thank you for your help. 2 replies	Aug '23
E	Evanston_Garneau Hey Josh, I am having trouble running the command ros2 run controller_manager spawner.py diff_cont it is saying that there is no executable found. I have installed and done everything the same as you up until that point. Any suggestions? From Evanston 3 replies	► Afshan_Navazdeen Aug '23
U	Uby I figure it out the problem. It was my linux system i just reinstall it an now it working fine	► Uby Aug '23
U	Uby Hello, I am driving the robot in gazebo with teleop twist keyboard but it not moving in rviz.	Aug '23
P	phil123456 was working fine launching the controllers manually but with the launcher <pre> spawner.py-5] [INFO] [1695724540.948537156] [spawner_diff_cont]: Waiting for /controller_manager services [spawner.py-6] [INFO] [1695724541.043151151] [spawner_joint_broad]: Waiting for /controller_manager services [gzclient -3] context mismatch in svg_surface_destroy [gzclient -3] context mismatch in svg_surface_destroy [spawner.py-5] [INFO] [1695724542.965812070] [spawner_diff_cont]: Waiting for /controller_manager services [spawner.py-6] [INFO] [1695724543.056949421] [spawner_joint_broad]: Waiting for /controller_manager services [spawner.py-5] [INFO] [1695724544.983464604] [spawner_diff_cont]: Waiting for /controller_manager services [spawner.py-6] [INFO] [1695724545.078081205] [spawner_joint_broad]: Waiting for /controller_manager services [spawner.py-5] [INFO] [1695724546.997859933] [spawner_diff_cont]: Waiting for /controller_manager services [spawner.py-6] [INFO] [1695724547.087982065] [spawner_joint_broad]: Waiting for /controller_manager services [spawner.py-5] [ERROR] [1695724549.01000553] [spawner_diff_cont]: Controller manager not available </pre>	Sep '23



Nguyen_Thao_Vy

Nov '20

Hi there, I am very thankful for your robot tutorial. Can you let me ask if skipping the tutorial section in video 12a will have much impact on subsequent tutorials. I don't really need great precision



Mohammed_Thabrez

► Evanston_Garneau Nov '23

Can anyone help out with this problem . I am also facing the same issue

1 reply



Mohammed_Thabrez

Nov '23

@JoshNewans can u help me with this

I download the entire articubot_one code from your Github and am running the command - ros2 launch articubot_one launch_sim.launch.py world:=src/articubot_one/worlds/obstacles.sdf

```
thabrez@thabrez-Inspiron-15-7000-Gaming:~/dev_ws2$ ros2 launch articubot_one launch_sim.launch.py
world:=src/articubot_one/worlds/obstacles.sdf
[INFO] [launch]: All log files can be found below /home/thabrez/.ros/log/2023-11-21-00-51-30-748484-thabrez-Inspiron-15-7000-Gaming-7568
[INFO] [launch]: Default logging verbosity is set to INFO
[ERROR] [launch]: Caught exception in launch (see debug for traceback): executable 'spawner.py' not found on the libexec directory
'/opt/ros/iron/lib/controller_manager'
```

I checked many times . used source install/setup.bash , correct directory too .
I have ros iron controller manager installed
The spawner.py file also exists in location opt/ros/iron/lib/controller_manager
Have done no modification to original code
newworld.sdf
Can u help me solve this ?? please

1 reply



mfekjaer

► Mohammed_Thabrez Nov '23

Hi @Mohammed_Thabrez

Looks like you are using ros2 iron, this tutorial is for foxy, but it is updated to humble, see https://www.youtube.com/watch?v=qoj5_fvBPil for the update

Try the humble branches of the github repositories.



hoonle135

► Mohammed_Thabrez Dec '23

Hi @JoshNewans

I'm currently facing some problems while working with ROS2 Foxy. I've followed the installation steps by executing the command:

```
sudo apt install ros-foxy-ros2-control ros-foxy-ros2-controllers ros-foxy-gazebo-ros2-control
```

However, when I attempt to launch the simulation using:

```
ros2 launch hoonle_bot launch_sim.launch.py world:=./src/hoonle_bot/worlds/obstacles.world
```

I encounter the following error:

```
[gzserver-2] [WARN] [1702871480.663104398] [gazebo_ros2_control]: Desired controller update period (0.0333333 s) is
slower than the gazebo simulation period (0.001 s).
[gzserver-2] [INFO] [1702871480.665948117] [gazebo_ros2_control]: Loaded gazebo_ros2_control.
[gzclient -3] context mismatch in svga_surface_destroy
[gzclient -3] context mismatch in svga_surface_destroy
[gzserver-2] [INFO] [1702871590.487554188] [controller_manager]: Loading controller 'diff_cont'
[gzserver-2] [INFO] [1702871590.717861782] [controller_manager]: Setting use_sim_time=True for diff_cont to match
controller manager (see ros2_control#325 for details)
[gzserver-2] [INFO] [1702871590.875489982] [controller_manager]: Configuring controller 'diff_cont'
[gzserver-2] [INFO] [1702871724.122437887] [controller_manager]: Configuring controller 'diff_cont'
[gzserver-2] [ERROR] [1702871724.122766325] [controller_manager]: Controller 'diff_cont' can not be configured from
'active' state.
```

Additionally, when I try to execute ros2 run control, I encounter the following error:

```
usage: ros2 run [-h] [--prefix PREFIX] package_name executable_name ...
ros2 run: error: the following arguments are required: executable_name, argv
```

Could you kindly assist me in resolving these issues?

Thank you for your help.

1 reply



Mohammed_Thabrez

► hoonle135 Dec '23

I FINALLY AND SUCCESSFULLY SOLVED ALL MY ISSUES AND WAS ABLE TO EXECUTE SLAM AND NAV2 ON BOTH SIMULATOR AND ALSO ON A REAL ROBOT THAT I BUILT .

I used ros Iron cause Foxy has reached its end of life (eol).

At the start of my project itself I was not able to download ros-foxy packages and i found that its dead .
check these :

- if u are actually able to download ros-foxy packages
- check for existence of ros2 FOXY
- check for gazebo and its packages
- try changing obstacles.world to a new obstacles2.sdf file (new extension from world to sdf)
- try running installation of controller packages command again
- try colcon build with symlink again
- download code directly from github rather than typing yourself

without the controller working u will get error in ros2 run control

1 reply



stevej

► Mohammed_Thabrez Dec '23

Hello Mohammed could you list the libraries and dependencies and drivers used to get your simulated robot to work as well as the physical robot...
thank you and congrats

1 reply



Mohammed_Thabrez

► stevej Dec '23

Sorry i didnt make a record of every package i downloaded .

I completely followed the articulated bot playlist and also watched the last video in the playlist that talks about Humble changes too .



WaelSonicS

► Phyo_Min_Thant Dec '23

Hey man thank you so much... It took me 4 hours debugging until I read your comment. You're genius man. Can I know how did you figured that out?



WaelSonics

you should remove the .py from spawner.py.
Check the update video in the series (Are you ready to Humble...)

► Evanston_Garneau Dec '23



jonnyr

Try:

```
ros2 run controller_manager spawner -h  
  
ros2 run controller_manager spawner diff_cont  
  
No .py
```

► Evanston_Garneau Jan '24



Umberto

Hi Josh,

Jan '24

@Josh: As many have said, much thanks for your great tutorials on ROS!

I'm building a robot based on your Articubot robot using 2 motors and a (ultrasound) sonar sensor (HCSR04). I am able to control the motors and drive around. But I'm stuck on how to correctly integrate the sonar sensor into ROS.

I've integrated the sonar sensor in joshnewans/ros_arduino_bridge example exposing distance information via the serial interface (via a "h" command, the distance as reported by the sonar sensor is returned)

Now I'm trying to create a hardware interface that publishes this sonar distance information in a ROS Topic sensor_msg/Range message. I followed the tutorial '12. Solving the problem EVERY robot has (with ros2_control)' and concluded that I need to create another State Interface (see <https://articulatedrobotics.xyz/media/assets/posts/mobile-robot/12-ros2-control/controllers.png>)

I tried to update joshnewans/diffdrive_arduino example to publishing the sensor_msg/Range message in diffdrive_arduino.cpp. But I am unable to do this since I cannot access ROS publish functions. Also reading further on the concepts of ROS, it seems not the correct way to do since diffdrive_arduino is for controlling the diff drive motors and NOT exposing a sonar sensor.

Then I tried to create another node that reads the serial information and publishes the Range message. But then I have the problem that you cannot use 1 serial interface by 2 nodes that each run as separate process.

So my question is:

How do you create 2 controllers exposing 2 hardware_interfaces (System & Sensor) that uses the same serial interface? Is there a way to share 1 serial interface between ROS nodes?

Regards,

Huub



nisat_av

Hi Josh,

At first, your tutorials are helping in a great course for my career and thanks for your tutorials.

Feb '24

I am trying to understand this ros2_control thing. but I am having the following issue when I am trying to load the controllers. btw I am using ros2-humble distro.

```
nisat@nt:~/mangobee_robot$ ros2 run controller_manager spawner diff_cont  
[INFO] [1706769460.539068927] [spawner_diff_cont]: Waiting for '/controller_manager' node to exist  
[INFO] [1706769462.551452191] [spawner_diff_cont]: Waiting for '/controller_manager' node to exist  
[INFO] [1706769464.564305671] [spawner_diff_cont]: Waiting for '/controller_manager' node to exist  
[INFO] [1706769466.575603025] [spawner_diff_cont]: Waiting for '/controller_manager' node to exist  
[ERROR] [1706769468.585726734] [spawner_diff_cont]: Controller manager not available  
[ros2run]: Process exited with failure 1
```

my_controllers.yaml file as follows:

```
controller_manager:  
  ros__parameters:  
    update_rate: 30  
    use_sim_time: true  
  
  diff_cont:  
    type: diff_drive_controller/DiffDriveController  
  
  joint_broad:  
    type: joint_state_broadcaster/JointStateBroadcaster
```

```
diff_cont:  
  ros__parameters:  
  
    publish_rate: 100.0  
    base_frame_id: base_link
```

```
    left_wheel_names: ['left_wheel_joint']  
    right_wheel_names: ['right_wheel_joint']  
    wheel_separation: 0.35  
    wheel_radius: 0.05
```

```
    use_stamped_vel: false
```

ros2_control.xacro file as follows,

```
<?xml version="1.0"?>  
<robot xmlns:xacro="http://www.ros.org/wiki/xacro">  
  
  <ros2_control name="GazeboSystem" type="system">  
    <hardware>  
      <plugin>gazebo_ros2_control/GazeboSystem</plugin>  
    </hardware>  
  
    <joint name="left_wheel_joint">  
      <command_interface name="velocity">  
        <param name="min">-10</param>  
        <param name="max">10</param>  
      </command_interface>  
      <state_interface name="position" />  
      <state_interface name="velocity" />  
    </joint>  
  
    <joint name="right_wheel_joint">  
      <command_interface name="velocity">  
        <param name="min">-10</param>  
        <param name="max">10</param>  
      </command_interface>  
      <state_interface name="position" />  
      <state_interface name="velocity" />  
    </joint>
```

```

        </command_interface>
        <state_interface name="velocity" />
        <state_interface name="position" />
    </joint>

    <gazebo>
        <plugin filename="libgazebo_ros2_control.so" name="gazebo_ros2_control">
            <parameters>$(find Mangobee_DifferentialDriveWheel_Robot)/config/my_controllers.yaml</parameters>
        </plugin>
    </gazebo>

</ros2_control>

</robot>

```

Please help me solve this issue.

Thank you
Nisath



cyberbilly

Feb '24

I am using ros2 iron and have "Contorller manager not available" errors. To make sure its not something I did, I downloaded articubot_one and it has the same issue.

First i fixed the .py issue to get this far:

```

diff_drive_spawner = Node(
    package="controller_manager",
    executable="spawner",
    arguments=["diff_cont"],
)

joint_broad_spawner = Node(
    package="controller_manager",
    executable="spawner",
    arguments=["joint_broad"],
)

```

Now when I enter

```
ros2 launch articubot_one launch_sim.launch.py world:=./src/my_bot/worlds/obstacles.world
```

Looks like I get two errors:

```
[ERROR] [gzserver-4]: process has died [pid 35946, exit code 255, cmd 'gzserver ./src/my_bot/worlds/obstacles.world -
slibgazebo_ros_init.so -slibgazebo_ros_factory.so -slibgazebo_ros_force_system.so --ros-args --params-file
/home/willli/shared/dev_ws/install/articubot_one/share/articubot_one/config/gazebo_params.yaml'].
```

Either the result or the real problem is the following:

```
[spawn_entity.py-4] [INFO] [1706795144.454419682] [spawn_entity]: Waiting for entity xml on robot_description
[spawn_entity.py-4] [INFO] [1706795144.465585204] [spawn_entity]: Waiting for service /spawn_entity, timeout = 30
[spawn_entity.py-4] [INFO] [1706795144.465941557] [spawn_entity]: Waiting for service /spawn_entity
[spawner-6] [INFO] [1706795146.204682358] [spawner_joint_broad]: Waiting for '/controller_manager' node to exist
[spawner-5] [INFO] [1706795146.405621127] [spawner_diff_cont]: Waiting for '/controller_manager' node to exist
[spawner-6] [INFO] [1706795148.218729921] [spawner_joint_broad]: Waiting for '/controller_manager' node to exist
[spawner-5] [INFO] [1706795148.418687173] [spawner_diff_cont]: Waiting for '/controller_manager' node to exist
[spawner-6] [INFO] [1706795150.234781109] [spawner_joint_broad]: Waiting for '/controller_manager' node to exist
[spawner-5] [INFO] [1706795150.429329085] [spawner_diff_cont]: Waiting for '/controller_manager' node to exist
[spawner-6] [INFO] [1706795152.248062370] [spawner_joint_broad]: Waiting for '/controller_manager' node to exist
[spawner-5] [INFO] [1706795152.447229883] [spawner_diff_cont]: Waiting for '/controller_manager' node to exist
[spawner-6] [ERROR] [1706795154.261474129] [spawner_joint_broad]: Controller manager not available
[spawner-5] [ERROR] [1706795154.459666959] [spawner_diff_cont]: Controller manager not available
[ERROR] [spawner-6]: process has died [pid 34051, exit code 1, cmd '/opt/ros/iron/lib/controller_manager/spawner
joint_broad --ros-args'].
[ERROR] [spawner-5]: process has died [pid 34042, exit code 1, cmd '/opt/ros/iron/lib/controller_manager/spawner
diff_cont --ros-args'].
[spawn_entity.py-4] [ERROR] [1706795174.516724938] [spawn_entity]: Service %s/spawn_entity unavailable. Was Gazebo
started with GazeboRosFactory?
[spawn_entity.py-4] [ERROR] [1706795174.517468743] [spawn_entity]: Spawn service failed. Exiting.
```

help

[1 reply](#)



cyberbilly

[► cyberbilly](#) Feb '24

So... I couldn't get past this error smurfing the innernut yielded notta. I then blew my package away and downloaded joshes to the same result... I then download an unrelated Ros2 to package from some other unrelated github and YIKES SAME same... Then I completely removed ros2-iron and installed ros2-humble and now everything is fine. It all works as specified. There is some issue/change in iron



Mohit Gupta

[► Ziad_Shoeib](#) Mar '24

Hi Ziad, I am facing the same issue. Did you find the solution?



Hasan Basari

[► Stress_Robotics](#) Apr '24

how would you solve this issue please explain clearly



Hasan Basari

[► Stress_Robotics](#) Apr '24

you can solve this issues, how would you solve this issue please explain clearly



saad_waqar

Apr '24

Hi [@JoshNewans](#)

Thank you so much for the hardwork.

I have been following and developing a physical robot as well.

I am facing an issue that:

When I shifted to ros2 control, the robot is driving in the opposite direction (in the Gazebo simulation), but when I was working with Gazebo diff_drive, it was running perfectly fine. Can anyone please guide me? I have been struggling with this for 2 days now.

I checked your response to another similar question, so I have similarly used the axis orientations for the wheels, it works good with joint_state_publisher and when working with gazebo_diff_drive, but when I use ros2 control, the wheels rotate inversely.

Can you please shed some light?

Thank you so much!

Thank you



Tim Hayes

[► JoshNewans](#) May '24

Hi [@JoshNewans](#), here is the error and the error message and how to fix it. I have been struggling with this for 2 days now. I checked your response to another similar question, so I have similarly used the axis orientations for the wheels, it works good with joint_state_publisher and when working with gazebo_diff_drive, but when I use ros2 control, the wheels rotate inversely.



baha_jouli

Jun '24

To change "l298" to "bts,7960b" what modifications are needed in the code?



ThurzinRB

Uby Jul '24

I don't know if it's the same issue. I was having the same error message: "Could not contact service /controller_manager/list_hardware_interfaces."

After some troubleshooting, I found that there was a mistake in my ros2_control.xacro file:

I had put the <gazebo> tag with its plugin inside the <ros2_control> tag when it needed to be placed separately.



Flamethrower

Aug '24

@JoshNewans Hello Josh. Thank you for all these tutorials. You may be aware of the issue where the controller_manager is apparently not available. Specifically, the output when launching is this:

```
[INFO] [launch]: All log files can be found below /home/flammethrower/.ros/log/2024-08-04-18-35-28-656714-robot-3619
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [robot_state_publisher-1]: process started with pid [3621]
[INFO] [gzserver-2]: process started with pid [3623]
[INFO] [gzclient -3]: process started with pid [3625]
[INFO] [spawn_entity.py-4]: process started with pid [3628]
[INFO] [spawner.py-5]: process started with pid [3631]
[INFO] [spawner.py-6]: process started with pid [3633]
[robot_state_publisher-1] Parsing robot urdf xml string.
[robot_state_publisher-1] Link FL_wheel had 0 children
[robot_state_publisher-1] Link FR_wheel had 0 children
[robot_state_publisher-1] Link RL_wheel had 0 children
[robot_state_publisher-1] Link RR_wheel had 0 children
[robot_state_publisher-1] Link chassis had 1 children
[robot_state_publisher-1] Link laser_frame had 0 children
[robot_state_publisher-1] [INFO] [1722789331.093667060] [robot_state_publisher]: got segment FL_wheel
[robot_state_publisher-1] [INFO] [1722789331.098568099] [robot_state_publisher]: got segment FR_wheel
[robot_state_publisher-1] [INFO] [1722789331.098785080] [robot_state_publisher]: got segment RL_wheel
[robot_state_publisher-1] [INFO] [1722789331.098873561] [robot_state_publisher]: got segment RR_wheel
[robot_state_publisher-1] [INFO] [1722789331.098906653] [robot_state_publisher]: got segment base_link
[robot_state_publisher-1] [INFO] [1722789331.098934339] [robot_state_publisher]: got segment chassis
[robot_state_publisher-1] [INFO] [1722789331.098960764] [robot_state_publisher]: got segment laser_frame
[spawner.py-5] [INFO] [1722789332.642764464] [spawner_diff_cont]: Waiting for /controller_manager services
[spawner.py-6] [INFO] [1722789332.753680821] [spawner_joint_broad]: Waiting for /controller_manager services
[spawn_entity.py-4] [INFO] [1722789333.814943066] [spawn_entity]: Spawn Entity started
[spawn_entity.py-4] [INFO] [1722789333.817251300] [spawn_entity]: Loading entity published on topic robot_description
[spawn_entity.py-4] [INFO] [1722789333.824613498] [spawn_entity]: Waiting for entity xml on robot_description
[spawn_entity.py-4] [INFO] [1722789333.840584852] [spawn_entity]: Waiting for service /spawn_entity, timeout = 30
[spawn_entity.py-4] [INFO] [1722789333.842771012] [spawn_entity]: Waiting for service /spawn_entity
[spawner.py-5] [INFO] [1722789334.688257733] [spawner_diff_cont]: Waiting for /controller_manager services
[spawner.py-6] [INFO] [1722789334.793537609] [spawner_joint_broad]: Waiting for /controller_manager services
[spawner.py-5] [INFO] [1722789336.754853755] [spawner_diff_cont]: Waiting for /controller_manager services
[spawner.py-6] [INFO] [1722789336.890463941] [spawner_joint_broad]: Waiting for /controller_manager services
[spawner.py-5] [INFO] [1722789338.799318653] [spawner_diff_cont]: Waiting for /controller_manager services
[spawner.py-6] [INFO] [1722789338.942930759] [spawner_joint_broad]: Waiting for /controller_manager services
[spawn_entity.py-4] [INFO] [1722789339.373300524] [spawn_entity]: Calling service /spawn_entity
[spawn_entity.py-4] [INFO] [1722789340.397202490] [spawn_entity]: Spawn status: SpawnEntity: Successfully spawned
entity [robot]
[gzserver-2] [WARN] [1722789340.508036852] [rcl]: Found remap rule '~/out:=scan'. This syntax is deprecated. Use '--ros-args --remap ~/out:=scan' instead.
[gzserver-2] [WARN] [1722789340.586504745] [rcl]: Found remap rule '~/out:=scan'. This syntax is deprecated. Use '--ros-args --remap ~/out:=scan' instead.
[INFO] [spawn_entity.py-4]: process has finished cleanly [pid 3628]
[gzserver-2] [INFO] [1722789340.836605840] [gazebo_ros2_control]: Loading gazebo_ros2_control plugin
[gzserver-2] [INFO] [1722789340.856579331] [gazebo_ros2_control]: Starting gazebo_ros2_control plugin in namespace: /
[gzserver-2] [INFO] [1722789340.860006190] [gazebo_ros2_control]: Starting gazebo_ros2_control plugin in ros 2 node: gazebo_ros2_control
[gzserver-2] [INFO] [1722789340.860619318] [gazebo_ros2_control]: Loading parameter files
/home/flammethrower/robot_ws/install/robot/share/robot/config/my_controllers.yaml
[spawner.py-5] [INFO] [1722789340.877581837] [spawner_diff_cont]: Waiting for /controller_manager services
[gzserver-2] [INFO] [1722789340.884953905] [gazebo_ros2_control]: connected to service!! robot_state_publisher
[gzserver-2] [INFO] [1722789340.893442322] [gazebo_ros2_control]: Recieved urdf from param server, parsing...
[gzserver-2] [INFO] [1722789340.948132774] [gazebo_ros2_control]: Loading joint: FL_wheel_joint
[gzserver-2] [INFO] [1722789340.948472162] [gazebo_ros2_control]: State:
[gzserver-2] [INFO] [1722789340.948567995] [gazebo_ros2_control]: velocity
[gzserver-2] [INFO] [1722789340.948771920] [gazebo_ros2_control]: position
[gzserver-2] [INFO] [1722789340.948853605] [gazebo_ros2_control]: Command:
[gzserver-2] [INFO] [1722789340.948888198] [gazebo_ros2_control]: velocity
[gzserver-2] [INFO] [1722789340.949095438] [gazebo_ros2_control]: Loading joint: FR_wheel_joint
[gzserver-2] [INFO] [1722789340.949171771] [gazebo_ros2_control]: State:
[gzserver-2] [INFO] [1722789340.949207993] [gazebo_ros2_control]: velocity
[gzserver-2] [INFO] [1722789340.949243345] [gazebo_ros2_control]: position
[gzserver-2] [INFO] [1722789340.949275752] [gazebo_ros2_control]: Command:
[gzserver-2] [INFO] [1722789340.949304863] [gazebo_ros2_control]: velocity
[gzserver-2] [INFO] [1722789340.949344659] [gazebo_ros2_control]: Loading joint: RL_wheel_joint
[gzserver-2] [INFO] [1722789340.949382381] [gazebo_ros2_control]: State:
[gzserver-2] [INFO] [1722789340.949414789] [gazebo_ros2_control]: velocity
[gzserver-2] [INFO] [1722789340.949445659] [gazebo_ros2_control]: position
[gzserver-2] [INFO] [1722789340.949479307] [gazebo_ros2_control]: Command:
[gzserver-2] [INFO] [1722789340.949507511] [gazebo_ros2_control]: velocity
[gzserver-2] [INFO] [1722789340.949543029] [gazebo_ros2_control]: Loading joint: RR_wheel_joint
[gzserver-2] [INFO] [1722789340.949577140] [gazebo_ros2_control]: State:
[gzserver-2] [INFO] [1722789340.949608121] [gazebo_ros2_control]: velocity
[gzserver-2] [INFO] [1722789340.949638640] [gazebo_ros2_control]: position
[gzserver-2] [INFO] [1722789340.949670269] [gazebo_ros2_control]: Command:
[gzserver-2] [INFO] [1722789340.949699621] [gazebo_ros2_control]: velocity
[gzserver-2] [INFO] [1722789340.951049691] [gazebo_ros2_control]: Loading controller_manager
[gzserver-2] terminate called after throwing an instance of 'std::system_error'
[gzserver-2] what(): Invalid argument
[gzserver-2] Aborted (core dumped)
[ERROR] [gzserver-2]: process has died [pid 3623, exit code 134, cmd 'gzserver -s libgazebo_ros_init.so -s libgazebo_ros_factory.so -s libgazebo_ros_force_system.so '].
```

I have tested this by downloading the project folder off of GitHub and the error persists. I'm pretty sure I've downloaded all of the necessary packages, so what might be the problem here? Thanks in advance.

1 reply



george_bethel

Sep '24



F Flamethrower:

Waiting

What ROS2 distro are you using?

Also, there seem to be an issue with starting the controller manager



AlePuglisi

Dec '24

Hey Josh, I love this series, It helped me learn ROS 2 with practical examples.

At the end of the `ros2_control` video, you introduce transmissions, as I understand, they are useful to set up a gearbox interface for the simulator. I'm working on a legged robot simulation and control with Gazebo Classic and ros2 humble, with controllers based on `ros2_control`.

I need to simulate the motor gearbox to make the simulation more reliable.

Do you know how to set up a reduction ratio properly? All the resources I found online seem draft and even the project that uses it is not working properly, setting a reduction ratio of 0.0 or a string as ratio is fine for the parser (it is completely ignored...).

Do you have some resources, examples or suggestions?



Golden

4 May

yo,

so I have following this tutorial till this part so far (video 12) and there is one issue that I am facing for a long time now with no any solution,

the issue is, when I run the gazebo simulation after adding the codes, the robot with the slightest touch when I am running `telop_twist_keyboards`, it just go ham, it moves there is no tomorrow, no control nothing, and it rotates like a washing machine without stopping in between, I played around with the yaml controllers but no change, I tried to send the vel instead by publishing the topic, and it still run ham, there is no change in the code from the original code, yet for some reason mine is crazy. I figured the spawn instead of `spawn.py` but there is nothing much so far.

I am using ros2 humble and ubuntu 22

this is the current ros2 plugin code

```
<ros2_control name="GazeboSystem" type="system">
  <hardware>
    <plugin>gazebo_ros2_control/GazeboSystem</plugin>
  </hardware>
  <joint name="left_wheel_joint">
    <command_interface name="velocity">
      <param name="min">-10</param>
      <param name="max">10</param>
    </command_interface>
    <state_interface name="velocity"/>
    <state_interface name="position"/>
  </joint>
  <joint name="right_wheel_joint">
    <command_interface name="velocity">
      <param name="min">-10</param>
      <param name="max">10</param>
    </command_interface>
    <state_interface name="velocity"/>
    <state_interface name="position"/>
  </joint>
</ros2_control>

<gazebo>
  <plugin name="gazebo_ros2_control" filename="libgazebo_ros2_control.so">
    <parameters>home/ubuntu/ros2_ws/src/articubot_one/my_controllers.yaml</parameters>
  </plugin>
</gazebo>
```

and this is the current .yaml controller

```
controller_manager:
  ros__parameters:
    update_rate: 30
    use_sim_time: true

diff_cont:
  type: diff_drive_controller/DiffDriveController

joint_broad:
  type: joint_state_broadcaster/JointStateBroadcaster

diff_cont:
  ros__parameters:

publish_rate: 30.0

base_frame_id: base_link

left_wheel_names: ['left_wheel_joint']
right_wheel_names: ['right_wheel_joint']
wheel_separation: 0.35
wheel_radius: 0.05

use_stamped_vel: false
```

please help me with this issue, anything or any clue will be helpful.

[Continue Discussion](#)



